

# The Mitnick Attack

April, 2026

## Contents

---

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Overview</b>                                         | <b>2</b>  |
| <b>2</b> | <b>Lab Environment Setup</b>                            | <b>2</b>  |
| 2.1      | Starting the Containers . . . . .                       | 2         |
| 2.2      | Configuring the .rhosts File on X-Terminal . . . . .    | 2         |
| <b>3</b> | <b>Task 1: Simulated SYN Flooding</b>                   | <b>3</b>  |
| 3.1      | Populating the ARP Cache . . . . .                      | 3         |
| 3.2      | Stopping the Trusted Server . . . . .                   | 4         |
| <b>4</b> | <b>Task 2: Spoof TCP Connections and rsh Sessions</b>   | <b>5</b>  |
| 4.1      | Task 2.1: Spoofing the First TCP Connection . . . . .   | 5         |
| 4.1.1    | Step 1: Spoofed SYN Packet . . . . .                    | 5         |
| 4.1.2    | Steps 2–3: Full Sniff-and-Spoof Attack Script . . . . . | 5         |
| 4.1.3    | Verifying Command Execution . . . . .                   | 6         |
| 4.2      | Task 2.2: Spoofing the Second TCP Connection . . . . .  | 7         |
| <b>5</b> | <b>Task 3: Setting Up a Backdoor</b>                    | <b>8</b>  |
| <b>6</b> | <b>Countermeasures</b>                                  | <b>10</b> |
| <b>7</b> | <b>Conclusion</b>                                       | <b>10</b> |
| <b>8</b> | <b>References</b>                                       | <b>10</b> |

## 1 Overview

---

The Mitnick Attack is a variation of a TCP session hijacking attack performed by Kevin Mitnick in 1994 against Tsutomu Shimomura. The attack exploits the trusted relationship between two machines and the predictability of TCP Initial Sequence Numbers (ISNs) of that era. This lab recreates the attack in a controlled Docker environment with three machines:

| Host           | IP Address | Role                                   |
|----------------|------------|----------------------------------------|
| X-Terminal     | 10.9.0.5   | Target machine                         |
| Trusted Server | 10.9.0.6   | Trusted host (impersonated)            |
| Attacker       | 10.9.0.1   | Launches all spoofed packets via Scapy |

Table 1: Lab environment hosts and roles.

The attack proceeds in three phases: (1) silencing the trusted server via simulated SYN flooding, (2) spoofing a full TCP/rsh session to execute a command on X-Terminal, and (3) planting a backdoor via the `.rhosts` file.

## 2 Lab Environment Setup

---

### 2.1 Starting the Containers

```
cd Labsetup
dcbuild
dcup
```

### 2.2 Configuring the `.rhosts` File on X-Terminal

The `.rhosts` file on X-Terminal is configured to allow the Trusted Server to execute remote commands without a password, replicating Shimomura's original setup:

```
# On X-Terminal (as seed user)
echo 10.9.0.6 > ~/.rhosts
chmod 644 ~/.rhosts
```

This is verified by running a remote command from the Trusted Server:

```
# On Trusted Server
su seed
rsh 10.9.0.5 date
```

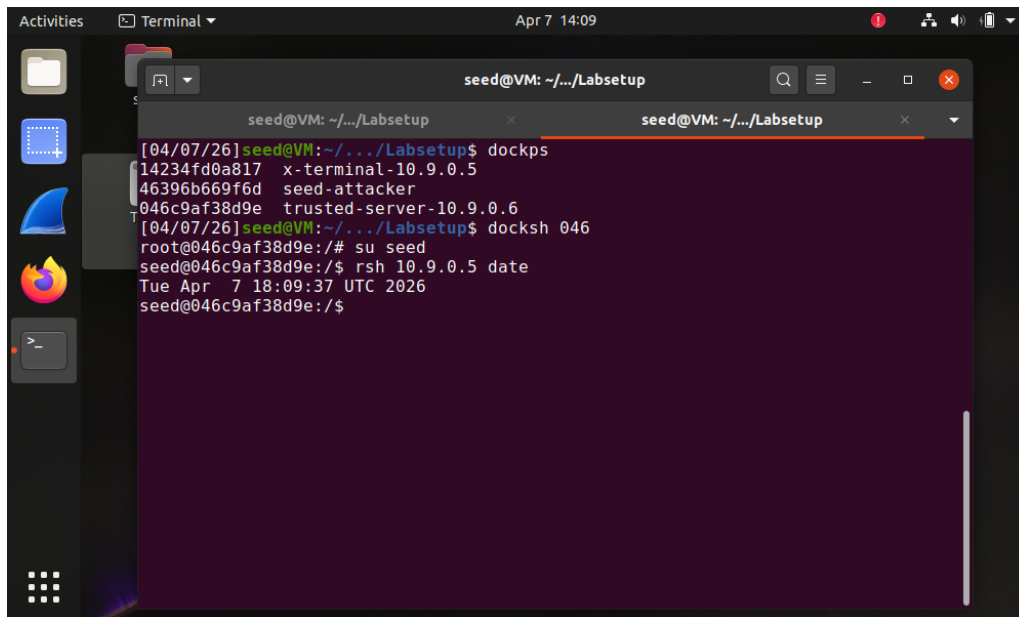


Figure 1: Successful passwordless `rsh date` from the Trusted Server to X-Terminal, confirming the `.rhosts` trust relationship is correctly configured.

### 3 Task 1: Simulated SYN Flooding

---

In the original attack, Mitnick launched a real SYN flood to silence the Trusted Server. Modern operating systems are immune to this, so we simulate it by stopping the container. However, before stopping it, X-Terminal must have the Trusted Server's MAC address in its ARP cache; otherwise it cannot send the SYN+ACK response to our spoofed packets.

#### 3.1 Populating the ARP Cache

We first ping the Trusted Server from X-Terminal to populate the ARP cache with the server's MAC address. After confirming connectivity, the ARP table is displayed and a permanent static ARP entry is added to associate the IP address 10.9.0.6 with the specified MAC address, ensuring that the mapping remains valid even if the server goes offline or dynamic ARP entries expire.

```

[04/07/26]seed@VM:~/.../Labsetup$ docksh 046
root@046c9af38d9e:/# su seed
seed@046c9af38d9e:/# rsh 10.9.0.5 date
Tue Apr 7 18:09:37 UTC 2026
seed@046c9af38d9e:/# exit
exit
root@046c9af38d9e:/# exit
exit
[04/07/26]seed@VM:~/.../Labsetup$ docksh 14
root@14234fd0a817:/# ping -c 1 10.9.0.6
PING 10.9.0.6 (10.9.0.6) 56(84) bytes of data.
64 bytes from 10.9.0.6: icmp_seq=1 ttl=64 time=0.074 ms

--- 10.9.0.6 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.074/0.074/0.074/0.000 ms
root@14234fd0a817:/# arp -n
Address          HWtype  HWaddress      Flags Mask    Iface
10.9.0.6         ether   02:42:0a:09:00:06 C              eth0
root@14234fd0a817:/# arp -s 10.9.0.6 02:42:0a:09:00:06
root@14234fd0a817:/# arp -n
Address          HWtype  HWaddress      Flags Mask    Iface
10.9.0.6         ether   02:42:0a:09:00:06 CM             eth0
root@14234fd0a817:/#

```

Figure 2: ARP table on X-Terminal showing a PERM entry for the Trusted Server's IP. This ensures X-Terminal can still send SYN+ACK packets after the server is taken offline.

### 3.2 Stopping the Trusted Server

```

docker stop 046c9af38d9e
docker ps # verify only 2 containers remain

```

```

exit
[04/07/26]seed@VM:~/.../Labsetup$ docksh 14
root@14234fd0a817:/# ping -c 1 10.9.0.6
PING 10.9.0.6 (10.9.0.6) 56(84) bytes of data.
64 bytes from 10.9.0.6: icmp_seq=1 ttl=64 time=0.074 ms

--- 10.9.0.6 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.074/0.074/0.074/0.000 ms
root@14234fd0a817:/# arp -n
Address          HWtype  HWaddress      Flags Mask    Iface
10.9.0.6         ether   02:42:0a:09:00:06 C              eth0
root@14234fd0a817:/# arp -s 10.9.0.6 02:42:0a:09:00:06
root@14234fd0a817:/# arp -n
Address          HWtype  HWaddress      Flags Mask    Iface
10.9.0.6         ether   02:42:0a:09:00:06 CM             eth0
root@14234fd0a817:/# exit
exit
[04/07/26]seed@VM:~/.../Labsetup$ docker stop 046
046
[04/07/26]seed@VM:~/.../Labsetup$ dockps
14234fd0a817  x-terminal-10.9.0.5
46396b669f6d  seed-attacker
[04/07/26]seed@VM:~/.../Labsetup$

```

Figure 3: `docker ps` output confirming the Trusted Server container is stopped. It is now effectively silenced, unable to send RST packets in response to any SYN+ACK it receives.

**Analysis:** Without the permanent ARP entry, X-Terminal would send an ARP request to find the Trusted Server's MAC before replying with SYN+ACK, and since the server is offline, no reply would come and the connection would never progress. The permanent ARP entry bypasses this.

## 4 Task 2: Spoof TCP Connections and rsh Sessions

### 4.1 Task 2.1: Spoofing the First TCP Connection

The first TCP connection replicates the rsh client side: the attacker impersonates the Trusted Server and completes a three-way handshake with X-Terminal, then sends rsh session data containing the command to execute.

#### 4.1.1 Step 1: Spoofed SYN Packet

We first verify that a spoofed TCP SYN packet triggers a SYN-ACK response from the X-Terminal. The script uses Scapy to craft an IP packet with a forged source address (10.9.0.6) and sends a TCP SYN segment from port 1023 to the target service on port 514, initiating a connection attempt to test how the system responds to spoofed traffic.

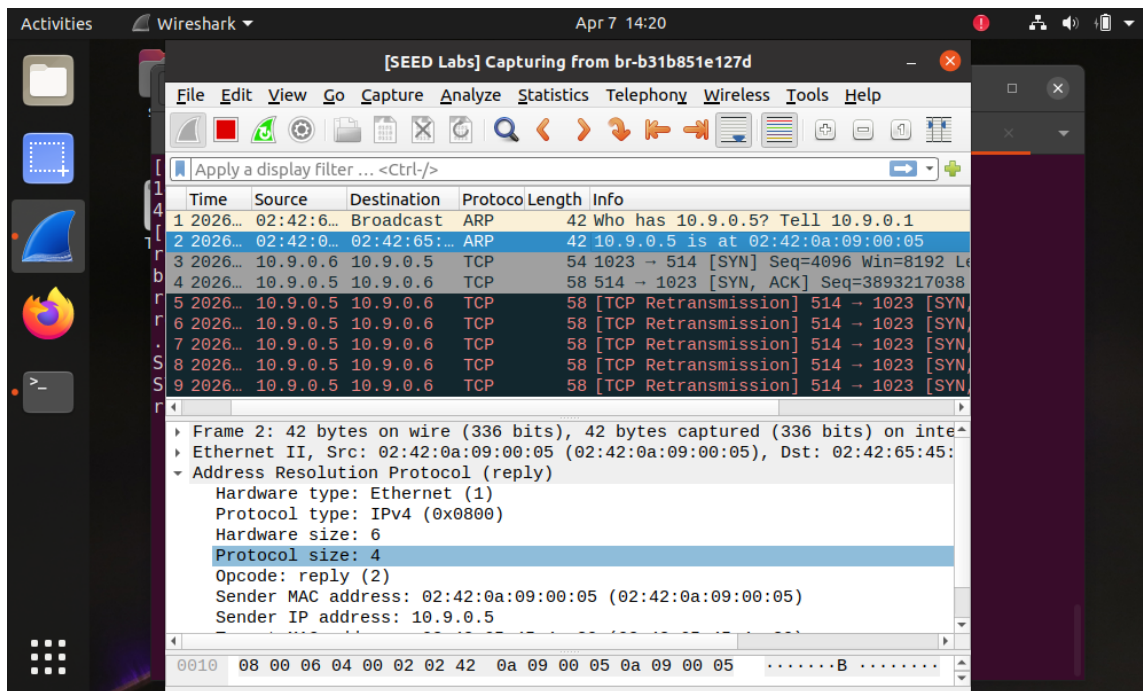


Figure 4: Wireshark capture showing the spoofed SYN packet (source 10.9.0.6, port 1023 → 514) followed by a SYN+ACK response from X-Terminal, confirming the server accepted the connection request from the impersonated Trusted Server.

#### 4.1.2 Steps 2–3: Full Sniff-and-Spoof Attack Script

The full attack script automates the entire exploitation sequence by coordinating both the primary and secondary TCP connections. It first launches a background listener thread to handle the second connection while simultaneously initiating the spoofed first connection to the target service. The script then sniffs the SYN+ACK response from the X-Terminal to extract its Initial Sequence Number (ISN), completes the TCP handshake using a forged ACK, and finally constructs and sends the rsh payload containing the command to be executed. In parallel, the listener responds to the secondary SYN request, completing the required handshake for that channel. Once both connections are successfully established and synchronized, the attack concludes by delivering the final command to the target system.

The rsh data payload has the format: `<stderr_port>\x00<uid>\x00<uid>\x00<cmd>\x00`, which tells `rshd` the port to use for the second (stderr) connection, the user identity, and the command to execute.

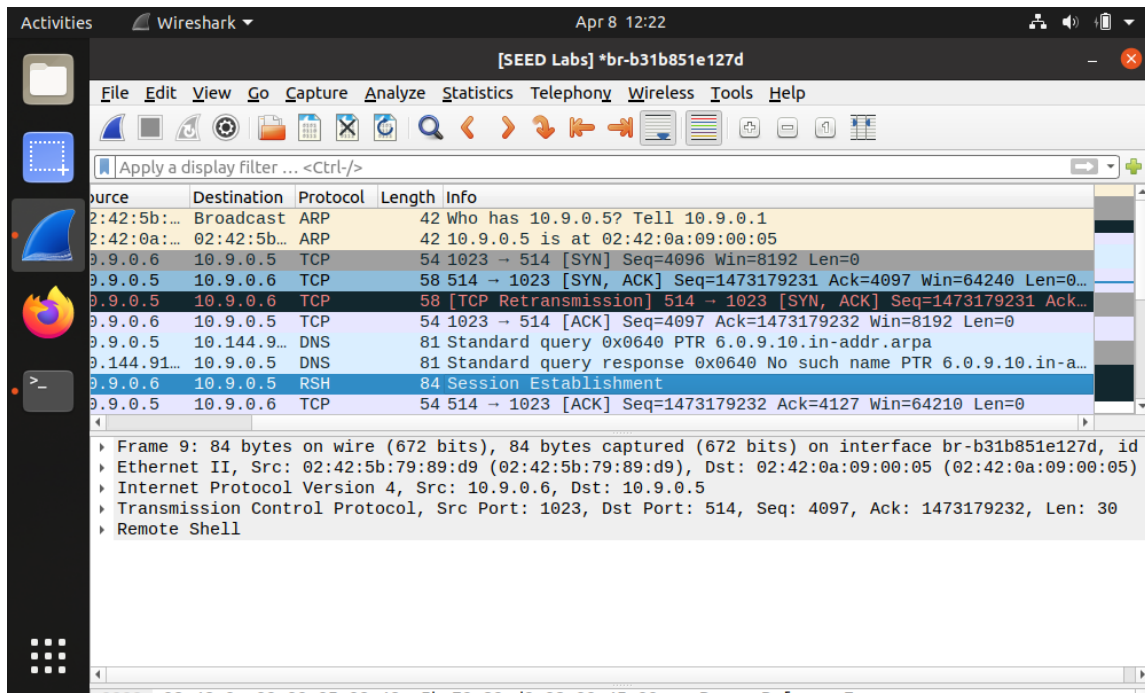


Figure 5: Wireshark capture of the complete first TCP connection: spoofed SYN (port 1023→514), X-Terminal's SYN+ACK, spoofed ACK completing the handshake, and the RSH data packet carrying the command payload.

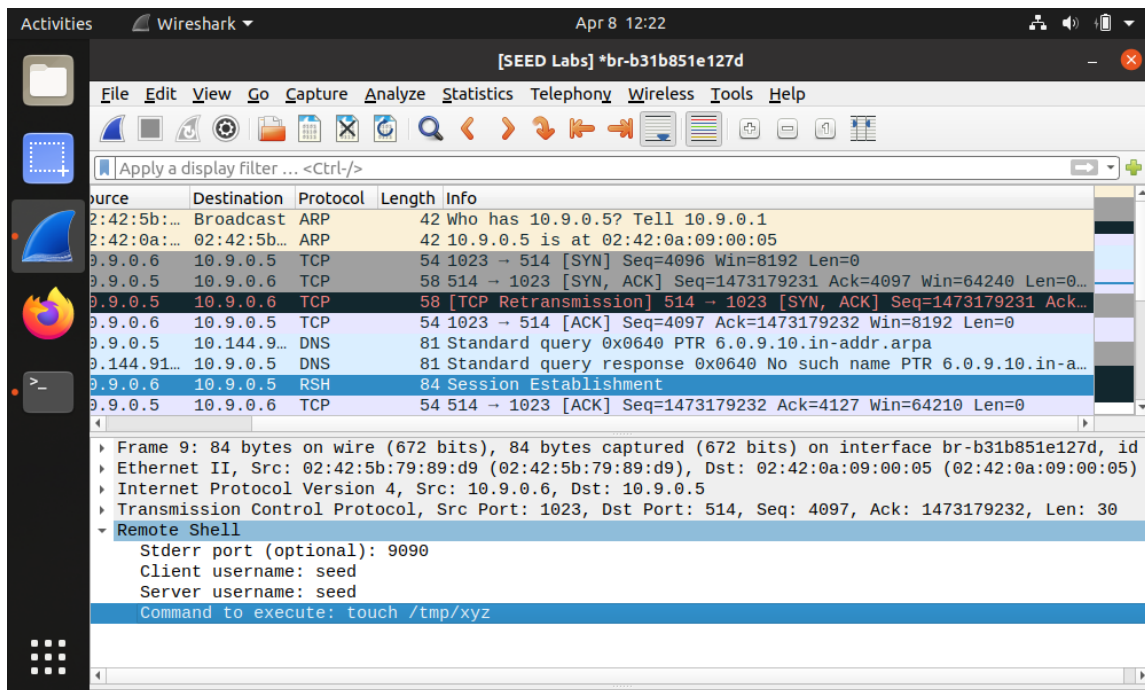


Figure 6: Wireshark packet details of the RSH data packet, showing stderr port 9090, client/server username `seed`, and the command `touch /tmp/xyz`.

### 4.1.3 Verifying Command Execution

```
# On X-Terminal
ls -l /tmp/xyz
```

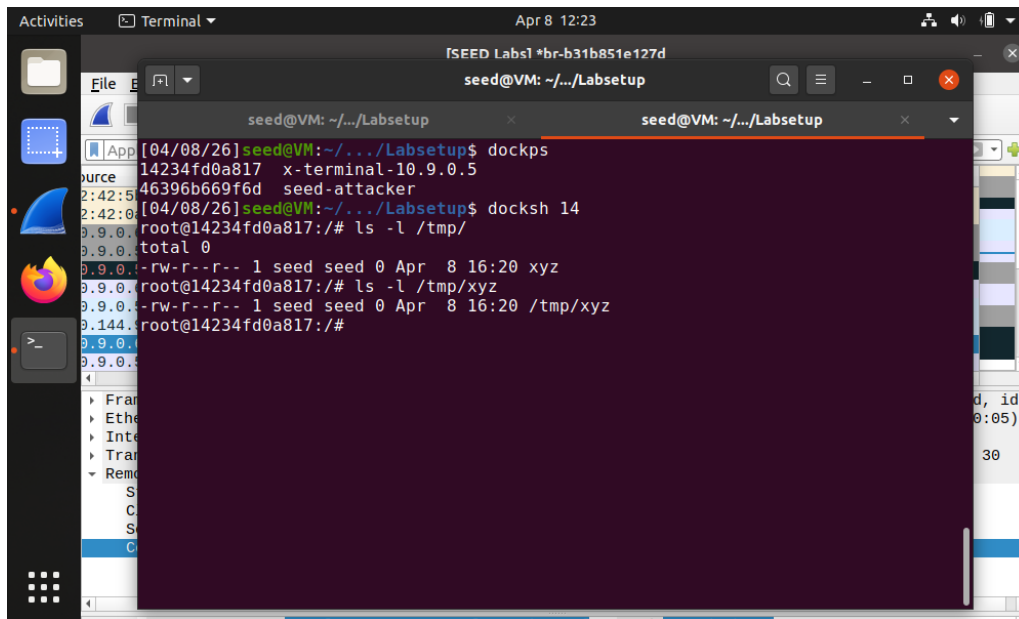


Figure 7: The file `/tmp/xyz` exists on X-Terminal, confirming that `rshd` successfully executed our spoofed command.

## 4.2 Task 2.2: Spoofing the Second TCP Connection

The `rshd` daemon requires a second TCP connection (used for stderr) to be established before it will execute any command. X-Terminal initiates this connection to port 9090 on the Trusted Server. Since the server is offline, the attacker's listener thread handles it by sniffing the SYN and replying with a spoofed SYN+ACK.

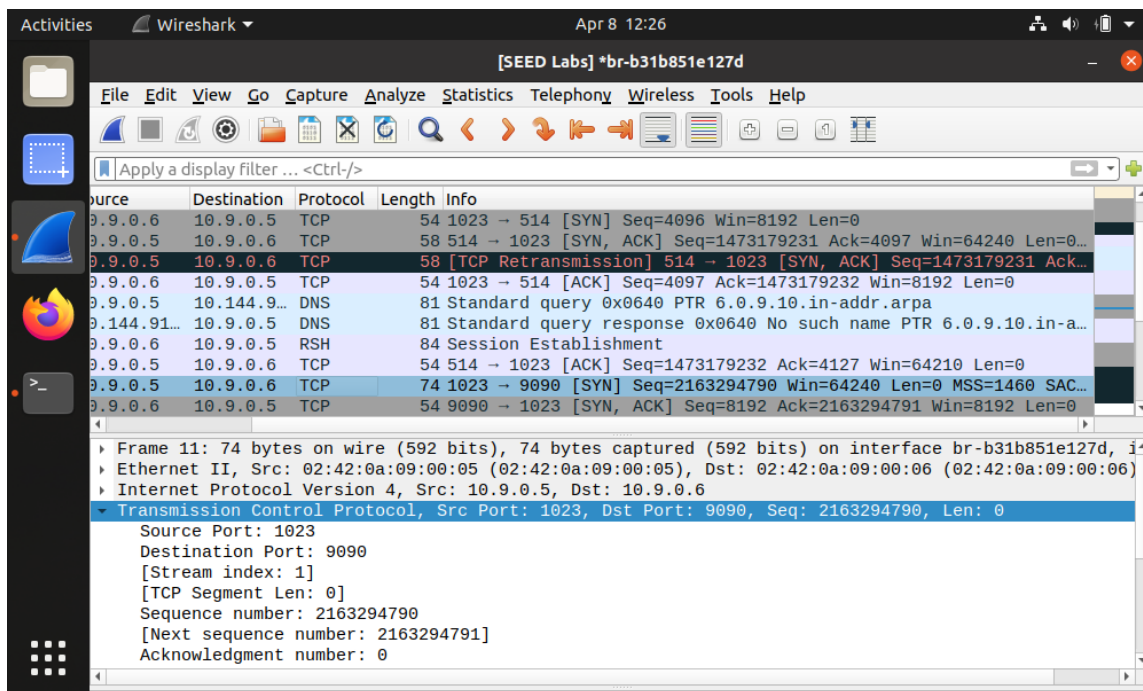


Figure 8: Wireshark showing the second TCP connection: X-Terminal sends a SYN to port 9090 on the Trusted Server's IP; the attacker replies with a spoofed SYN+ACK completing the handshake and unblocking `rshd`.



**Analysis:** The second connection is a mandatory gate in the rsh protocol. Without it `rshd` blocks indefinitely waiting for the stderr channel. By sniffing the SYN and replying with a spoofed SYN+ACK the attacker satisfies this requirement without a real server.

## 5 Task 3: Setting Up a Backdoor

Running a single command via the attack is effective but tedious to repeat. Mitnick's solution was to overwrite the `.rhosts` file with `+` `+`, which instructs `rshd` to trust *any* IP address with no password. We repeat the attack with the command changed:

```
COMMAND = "echo + + > .rhosts"
```

After running the modified script, the `.rhosts` file on X-Terminal is overwritten:

```
# On X-Terminal
cat /home/seed/.rhosts
+ +
```

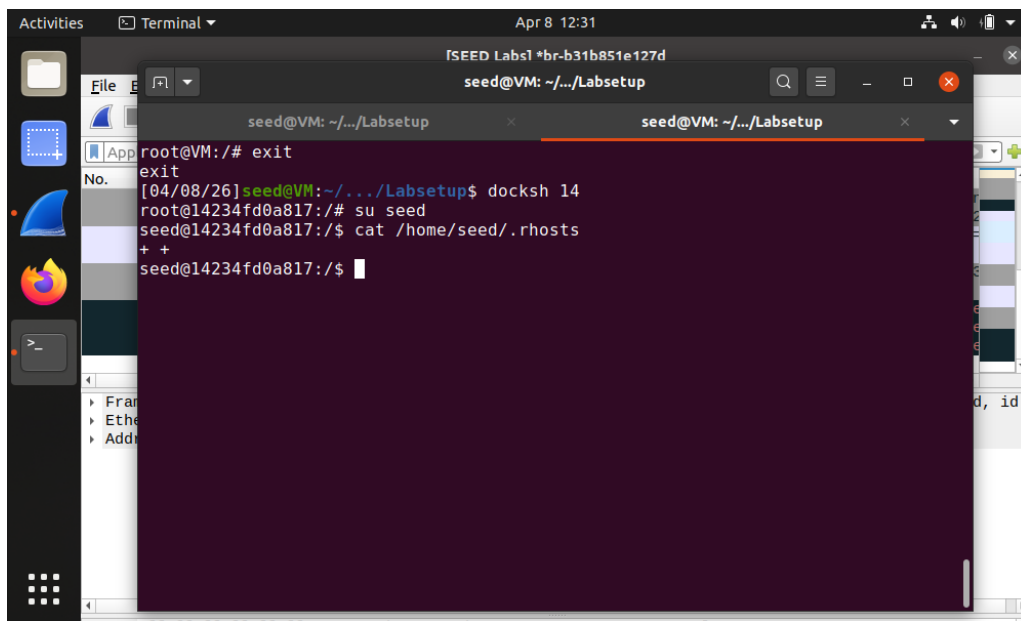


Figure 9: `cat .rhosts` on X-Terminal showing `+` `+`, confirming the backdoor command was executed by `rshd`.

The attacker can now log into X-Terminal from any machine with no password:

```
# On attacker
rsh 10.9.0.5
```



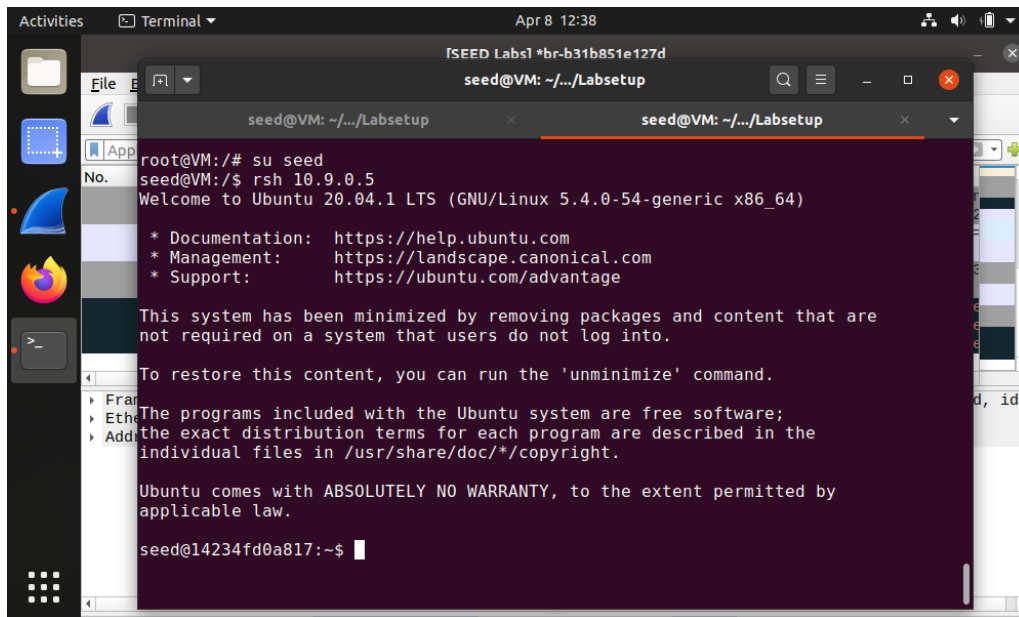


Figure 10: Passwordless `rsh` login to X-Terminal from the attacker machine. No password prompt appears because `.rhosts` now contains `+`, granting access to all hosts.

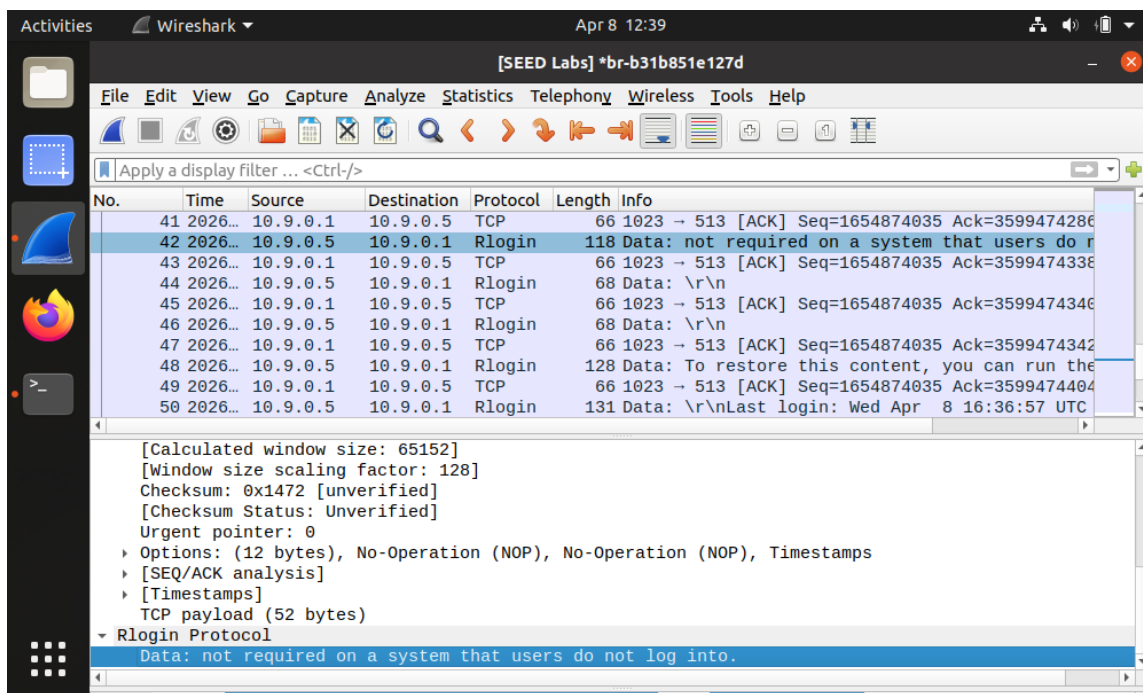


Figure 11: Wireshark capture of the backdoor `rsh` session showing Rlogin protocol packets between the attacker (10.9.0.1) and X-Terminal (10.9.0.5) with no authentication challenge.

## 6 Countermeasures

---

| Countermeasure             | Mechanism                         | Addresses           |
|----------------------------|-----------------------------------|---------------------|
| Randomized ISNs (RFC 6528) | Random ISN generation             | Sequence prediction |
| SSH instead of rsh/rlogin  | Cryptographic auth + encryption   | IP spoofing         |
| Ingress/Egress filtering   | Drop spoofed source IPs at router | IP spoofing         |
| Disable .rhosts            | Remove IP-based trust entirely    | Trust exploitation  |

Table 2: Summary of countermeasures against the Mitnick Attack.

**Randomized ISNs** are the most direct fix: without predictable sequence numbers, the attacker cannot craft a valid ACK blindly. **SSH** eliminates the entire attack surface by replacing IP-based trust with cryptographic authentication. **Ingress filtering** prevents spoofed packets from traversing network boundaries. **Removing .rhosts** eliminates the trust relationship that makes the attack worthwhile in the first place.

## 7 Conclusion

---

Mitnick’s 1994 Attack gave a valuable lesson in Cybersecurity. By silencing the Trusted Server, predicting (or sniffing) TCP sequence numbers, and injecting spoofed packets, we established a forged rsh session and executed arbitrary commands on X-Terminal — ultimately planting a + backdoor that granted permanent passwordless access.

The attack highlights three fundamental weaknesses that coexisted in 1994: non-random TCP ISNs, IP-address-based authentication (**.rhosts**), and the absence of encrypted remote shell protocols. All three have been addressed in modern systems (RFC 6528, SSH, ingress filtering), making a real-world Mitnick Attack essentially infeasible today.

## 8 References

---

1. SEED Labs: The Mitnick Attack Lab Description, Wenliang Du.
2. RFC 6528: Defending against Sequence Number Attacks.
3. Wenliang Du, *Computer & Internet Security: A Hands-on Approach*, 2nd Edition, Chapter 16.
4. Tsutomu Shimomura, *Takedown*, Hyperion, 1996.